Python Engineering

WEEK 03 - Modules & Coordinates | CLASS 09 - Modules & Imports | EX17

Preview: este ejercicio empezara oficialmente cuando cerremos EX16b. EX17 abre una nueva semana y una nueva clase.

Ruta que crearemos

```
01_python_bootcamp/
`-- week_03_modules_and_coordinates/
    `-- class_09_modules_imports/
        |-- notes/
        |-- exercises/
        |   `-- ex17_vertical_motion_modules/
        |       |-- main.py
        |       |-- vertical_motion.py
        |       `-- data_processing.py
        `-- mini_project/
```

Objetivo

Dejar de tener todo el programa en un unico archivo. Separaremos el simulador, el procesado de datos y el programa principal en modulos propios y aprenderemos a importarlos.

1. vertical_motion.py

Moveremos a este modulo las funciones fisicas/numericas que ya conoces:

```
validate_inputs(...)
euler_step(...)
simulate_vertical_motion(...)
analytical_solution(...)
calculate_errors(...)
```

2. data_processing.py

Crearemos una funcion sencilla que reciba trajectory_array y config y devuelva los resultados de procesado que ya has usado en EX16.

```
def process_trajectory(trajectory_array, config):
    ...
    return processing_results, processed_data
```

processing_results puede ser un diccionario con conteos, medias, desviaciones, indices y comprobacion de dt.

3. main.py

main.py sera el coordinador: define config, llama a los modulos y presenta resultados. No deberia contener las ecuaciones de Euler ni el detalle del filtrado.

```
from vertical_motion import (
    validate_inputs,
    simulate_vertical_motion,
    analytical_solution,
    calculate_errors
)

from data_processing import process_trajectory
```

4. Que significa import?

Sintaxis	Que hace	Ejemplo de uso
import module	importa el modulo completo	module.function()
import module as alias	importa con nombre corto	np.array(...)

Sintaxis	Que hace	Ejemplo de uso
from module import function	importa un nombre concreto	function(...)

5. __name__ == '__main__'

Usaremos por primera vez el main guard. Permite que un archivo se comporte de forma distinta cuando se ejecuta directamente y cuando se importa como modulo.

```
def main():
    ...

if __name__ == '__main__':
    main()
```

Cuando ejecutas main.py directamente, la condicion es True. Cuando otro archivo lo importa, el bloque no se ejecuta automaticamente.

6. Tu mision en EX17

```
1. Create the three files.
2. Move existing functions without changing their mathematics.
3. Import simulation functions into main.py.
4. Create process_trajectory() in data_processing.py.
5. Import and call process_trajectory() from main.py.
6. Keep config in main.py.
7. Add a main() function.
8. Add if __name__ == '__main__': main().
9. Run main.py from the terminal.
10. Verify results match EX16.
```

7. Regla del ejercicio

No cambiaremos la fisica ni el metodo numerico. El aprendizaje es puramente de arquitectura: **separar responsabilidades y reutilizar codigo entre archivos.**

Criterio de cierre

EX17 estara completado cuando main.py pueda ejecutar toda la simulacion importando codigo desde tus propios modulos y puedas explicar la diferencia entre importar un modulo completo e importar una funcion concreta.